



PONENCIA CLOUD NATIVE

Si no lo mides, no lo controlas

Prometheus en 4 piezas



ESCANEA Y ENTRA

La app

<http://34.55.181.146>

Créate un usuario y busca un
Pokémon.

Usuario admin / 123, o el tuyo.



SI TE PICA LA CURIOSIDAD

Prometheus

<http://34.10.208.11>

Las métricas crudas: targets, PromQL
y alertas,
moviéndose en vivo con lo que
ustedes generen.



SI TE PICA LA CURIOSIDAD

Grafana

<http://34.172.120.142>

El tablero, en solo lectura.

Las mismas métricas, ya dibujadas.

Las 2 de la madrugada

- Todo “verde”... pero el dashboard en blanco.
- ¿La app está sana? ¿Va a caer? ¿Ya cayó?
- **Monitoreo** (¿está vivo?) ≠ **observabilidad** (¿qué pasa **dentro**?).
- Contenedores efímeros: nacen y mueren en segundos. No puedes “entrar a mirar”.

Prometheus: proyecto **graduado** de la CNCF — el 2.º después de Kubernetes.

¿Suena complejo? Son sólo 4 piezas.



Exporters

¿De dónde salen las métricas?

- Un exporter expone métricas en HTTP, normalmente `/metrics`, en texto plano.
- **Modelo pull, no push:** Prometheus **va a buscar** los datos.
- Hay exporters para todo: SO (`node_exporter`), bases de datos, apps propias.

EN LA DEMO `localhost:9100/metrics` → `nombre{etiquetas} valor`

La app que vamos a observar

pokeapi — una pokédex web en Rust + Leptos. La van a usar ustedes.

- Buscas un pokémon: la 1.^a vez sale de la PokeAPI pública; las siguientes, del **caché en Redis** — y se nota en la latencia.
- Login, registro y roles (`ADMIN` / `EDITOR` / `VISITOR`).
- Ella misma expone `/metrics`: un **exporter hecho en casa**.

MongoDB

los usuarios

Redis

sesiones y caché de fichas

PokeAPI pública

el origen de los datos

No es un exporter de juguete: es **tu** app instrumentada. Eso es lo que harás en el trabajo.

El servidor

El recolector

- **Scrape:** “¿cuánto vale esto ahora?” cada 15 s.
- **Series temporales:** cada valor con su marca de tiempo.
- **Targets:** la lista de endpoints = el “cableado” (`prometheus.yml`).

EN LA DEMO

localhost:9090/targets → target **UP**

El cable (prometheus.yml)

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'pokeapi'          # nuestra app, un target más
    static_configs:
      - targets: ['pokeapi:3000']
```

“Ve a estas direcciones cada 15 s.” Eso es todo.

PromQL

Preguntarle a tus datos

De lo simple a lo útil en 4 pasos:

- | | | |
|---|---|--------------------------------------|
| 1 | <code>node_memory_MemAvailable_bytes</code> | el valor crudo |
| 2 | <code>node_cpu_seconds_total{mode="idle"}</code> | filtrar con etiquetas |
| 3 | <code>rate(node_cpu_seconds_total{mode="system"}[1m])</code> | <code>rate()</code> : qué tan rápido |
| 4 | <code>100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)</code> | % de uso de CPU |

`nombre` + `{etiquetas}` + `rate()` + agregación = el 80% del día a día.

Las mismas 4 herramientas, sobre la app

- › `sum(rate(pokeapi_http_peticiones_total[1m])) by (rol)` tráfico por rol
- › `sum(rate(pokeapi_pokemon_consultas_total[5m])) by (origen)` ¿caché o API pública?
- › `pokeapi_sesiones_activas` cuánta gente hay dentro
- › `sum(increase(pokeapi_login_errores_total[1m]))` logins fallidos

Cambia la app; el lenguaje, no. El **negocio** se pregunta igual que el CPU.

Alertmanager

Que el sistema te avise

- Una **regla de alerta** = una query de PromQL + un umbral + `for`.
- **Alertmanager** recibe, agrupa, silencia y enruta (Slack, email, PagerDuty).

```
- alert: CPUAlto
  expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100) > 80
  for: 1m
```

El `expr` es la misma query de la Pieza 3. Nada nuevo.

EN LA DEMO localhost:9090/alerts → FIRING → llega a localhost:9093

Las alertas de la app

PokeapiCaida	la app no responde al scrape (target DOWN)
PokeapiSinRedis	perdió el caché y las sesiones
PokeapiSinMongo	perdió la base de usuarios
PokeapiFuerzaBrutaDemo	demasiados passwords incorrectos por minuto

```
- alert: PokeapiFuerzaBrutaDemo
  expr: >-
    sum(increase(pokeapi_login_errores_total{motivo="password_incorrecto"}[1m]))
    > 5
  for: 30s
```

Hasta una alerta de **seguridad** es lo mismo: una pregunta, un umbral y un `for`.

Recap — las 4 piezas



Y encima: **Grafana** (dashboards), **OpenTelemetry** (instrumentación, ya graduado en la CNCF).

AHORA, USTEDES

Entren a la app

<http://34.55.181.146>

Usuario `admin / 123`, o regístrate con lo que quieras.
Busca pokémon. Falla el login a propósito. Rompe algo.

La demo en 4 actos · 10 min



Todo lo que están viendo lo generaron **ustedes** en los últimos 5 minutos.

Para seguir

DOCUMENTACIÓN OFICIAL

prometheus.io

EXPORTERS

`node_exporter` y exporters de la comunidad

VISUALIZACIÓN

Grafana

EN KUBERNETES

busca **kube-prometheus-stack**



“Si no lo mides,
no lo controlas.”

Ahora ya saben cómo medirlo.